University of Geneva

Faculty of Science

Exercise #1

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet

2026-03-09

1 Correlation of two random variables

- Let A and B be two independent random variables such that $A \sim \mathcal{N}(\mu_A, \sigma_A)$ and $B \sim \mathcal{N}(\mu_B, \sigma_B)$, with $\mu_A = \mu_B = 0$, $\sigma_A = 1$ and $\sigma_B = 2$.
- Let X and Y be two random variables such that $X = A + 4B$ and $Y = 2A + B$.

1. Derive analytically the following quantities:

(a) μ_X, μ_Y, σ_X and σ_Y, the expectations and standard deviations of X and Y.

- $$\begin{aligned}\mu_X &= \mathbb{E}[X] \\ &= \mathbb{E}[A + 4B] \\ &= \mathbb{E}[A] + 4\mathbb{E}[B] \\ &= \mu_A + 4\mu_B \\ &= 0\end{aligned}$$
- $$\begin{aligned}\mu_Y &= \mathbb{E}[Y] \\ &= \mathbb{E}[2A + B] \\ &= 2\mathbb{E}[A] + \mathbb{E}[B] \\ &= 2\mu_A + \mu_B \\ &= 0\end{aligned}$$
- $$\begin{aligned}\sigma_X^2 &= \mathrm{Var}[X] \\ &= \mathrm{Var}[A + 4B] \\ &= \mathrm{Var}[A] + 4^2\,\mathrm{Var}[B] \\ &= \sigma_A^2 + 4^2\sigma_B^2 \\ &= 1 + 16 \cdot 2^2 \\ &= 65\end{aligned}$$

  So $\sigma_X = \sqrt{65}$
- $$\begin{aligned}\sigma_Y^2 &= \mathrm{Var}[Y] \\ &= \mathrm{Var}[2A + B] \\ &= 2^2\,\mathrm{Var}[A] + \mathrm{Var}[B] \\ &= 4\sigma_A^2 + \sigma_B^2 \\ &= 4 + 4 \\ &= 8\end{aligned}$$

  So $\sigma_Y = \sqrt{8} = 2\sqrt{2}$

(b) The covariance between X and Y, defined as:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$$

- $\begin{aligned}\mathrm{Cov}(X,Y) &= \mathbb{E}[(X-\mu_X)(Y-\mu_Y)] \\ &= \mathbb{E}[XY] \\ &= \mathbb{E}[(A+4B)(2A+B)] \\ &= \mathbb{E}[2A^2+AB+8AB+4B^2] \\ &= 2\mathbb{E}[A^2]+9\mathbb{E}[AB]+4\mathbb{E}[B^2] \\ &= 2\mathbb{E}[A^2]+4\mathbb{E}[B^2] \qquad \text{Independance} \implies \mathbb{E}[AB] \Leftrightarrow \mathbb{E}[A]\mathbb{E}[B] \\ &= 2(\mathrm{Var}(A)+\mathbb{E}[A]^2)+4(\mathrm{Var}(B)+\mathbb{E}[B])^1 \\ &= 2(\sigma_A^2+\mu_A^2)+4(\sigma_B^2+\mu_B) \\ &= 2\sigma_A^2+4\sigma_B^2 \\ &= 18\end{aligned}$

(c) The correlation coefficient between X and Y, defined as:

$$\mathrm{Cor}(X,Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

- $\begin{aligned}\mathrm{Cor}(X,Y) &= \frac{\mathrm{Cov}(X,Y)}{\sigma_X\sigma_Y} \\ &= \frac{18}{\sqrt{65}\cdot 2\sqrt{2}} \\ &\approx 0.789\end{aligned}$

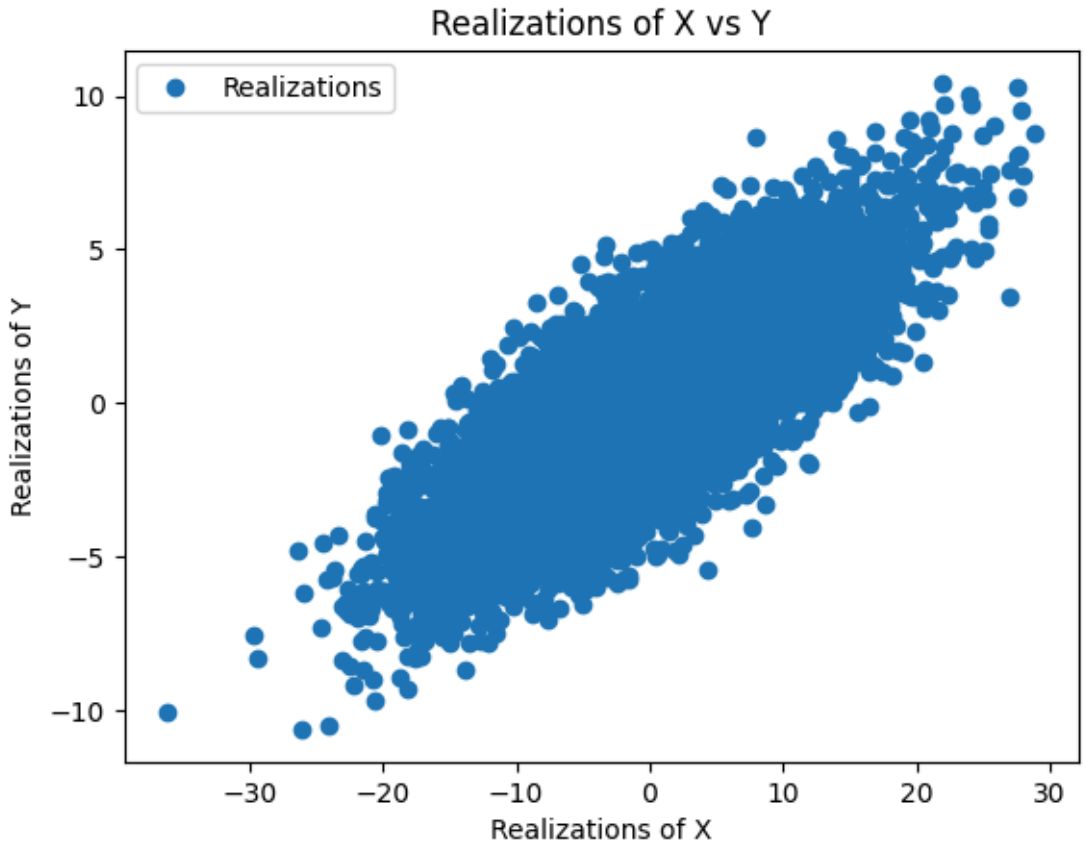
2. Using the language of your choice:

(a) Simulate $n = 10000$ realizations of A and B and draw the graph of the corresponding realizations of X vs realizations of Y.

```python
import numpy as np
import matplotlib.pyplot as plt

# Generate n realizations

np.random.seed(0)
n = 10000
mu_a, sig_a = 0, 1
mu_b, sig_b = 0, 2
a = np.random.normal(mu_a, sig_a, n)
b = np.random.normal(mu_b, sig_b, n)
x = a + 4 * b
y = 2 * a + b

z = np.linspace(np.min(x), np.max(x), 200)

# Plot results
plt.figure()
plt.plot(x, y, 'o', label='Realizations')
plt.xlabel("Realizations of X")
plt.ylabel("Realizations of Y")
```

[1] $\mathrm{Var}(U) = \mathbb{E}[U^2] - \mathbb{E}[U]^2 \Longleftrightarrow E[U^2] = \mathrm{Var}(U) + \mathbb{E}[U]^2$

```
plt.title("Realizations of X vs Y" )
plt.legend()
plt.show()
```



(a) How does this relate to the value of $\text{Cor}(X, Y)$? Is the slope equal to the correlation coefficient? Comment on this last point.

The slope can be obtained by computing $\frac{\text{Cov}(X,Y)}{\text{Var}(X)}$ whereas the correlation coefficient can be obtained by computing $\frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$.

So as $\sigma_X^2 = 65 \neq \sigma_X \sigma_Y = \sqrt{65}2\sqrt{2}$, the slope is not equal to the correlation coefficient.

(b) Compute the empirical correlation coefficient from the n realizations and compare it to the value of $\text{Cor}(X, Y)$ obtained from the above analytical derivation (1c).

```
print(f"Empirical correlation coefficient Cor(X, Y): {np.corrcoef(x, y)[0,
1]}")
```

```
Empirical correlation coefficient Cor(X, Y): 0.7883387100133817
```

and $\text{Cor}(X, Y) \approx 0.789$ from analytical derivation (1c).

So correlation coefficient are approximately the same between empirical correlation coefficient and analytical coefficient.

2 Hidden sine

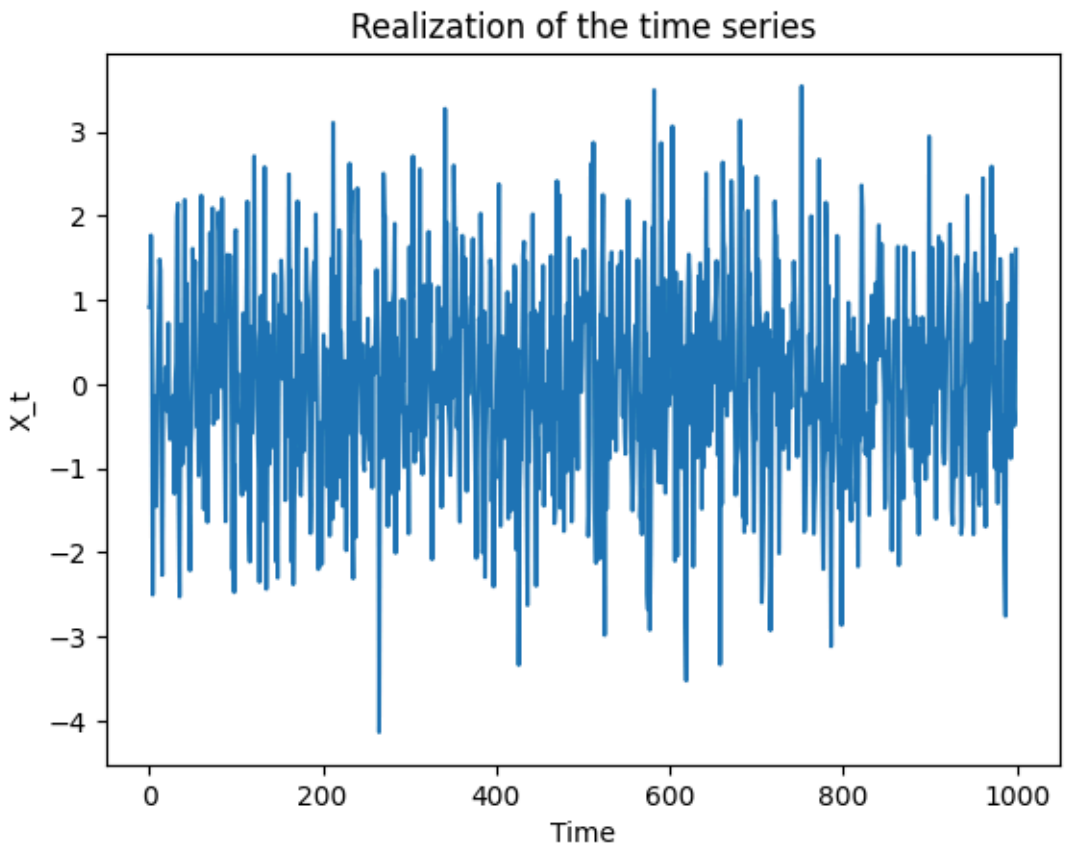
Consider the following time series:

$$\forall t \in \{1, ..., n\}, \quad X_t = \sin\left(\frac{2\pi t}{T}\right) + \varepsilon_t$$

with $\varepsilon_t \sim \mathcal{N}(0, 1)$, $T = 10$ and $n = 1000$.

1. Plot a realization of the time series above.

```
X_t = lambda t, T: np.sin(2 * np.pi * t / T) + np.random.normal(0, 1,
size=len(t))
X = X_t(np.arange(1, 1001), 10)

plt.figure()
plt.plot(X)
plt.xlabel("Time")
plt.ylabel("X_t")
plt.title("Realization of the time series")
plt.show()
```



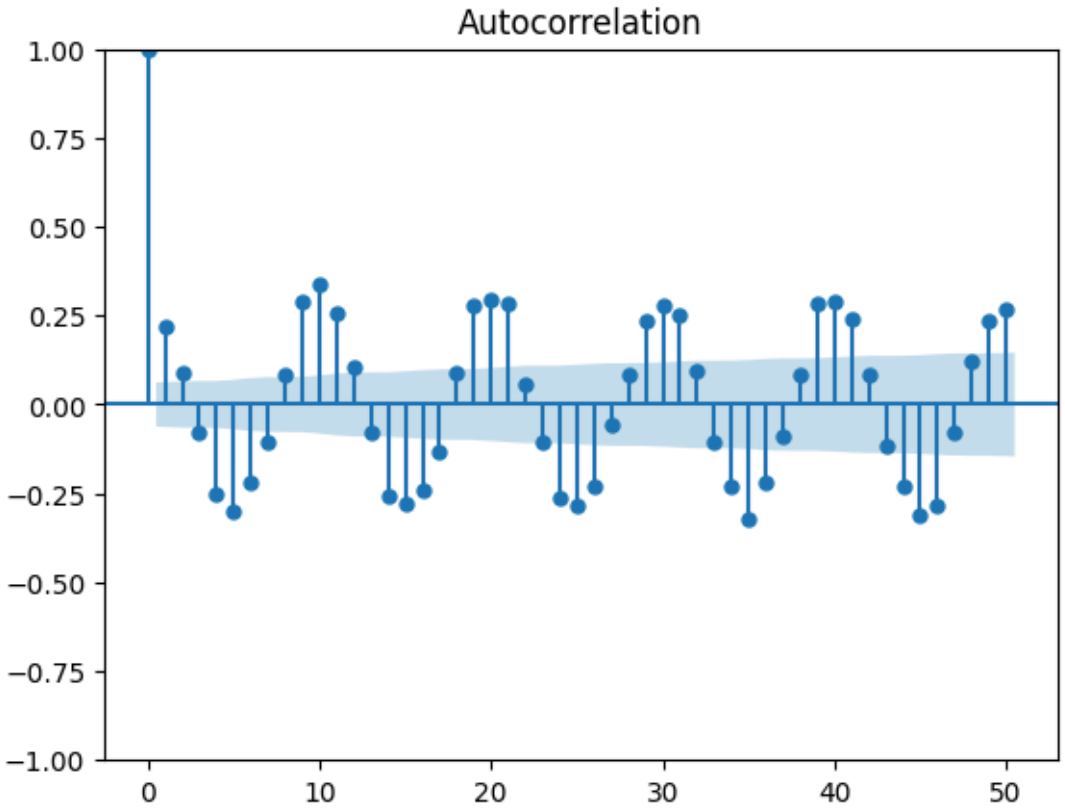
1. Compute and plot the corresponding ACF graph for lags 0 to 50.

The Autocorrelation Function is defined as the correlation between the variable and itself with a shift of τ:

$$\mathrm{Cor}(X_t, X_{t+\tau}) = \frac{\mathrm{Cov}(X_t, X_{t+\tau})}{\sigma_t \sigma_{t+\tau}}$$

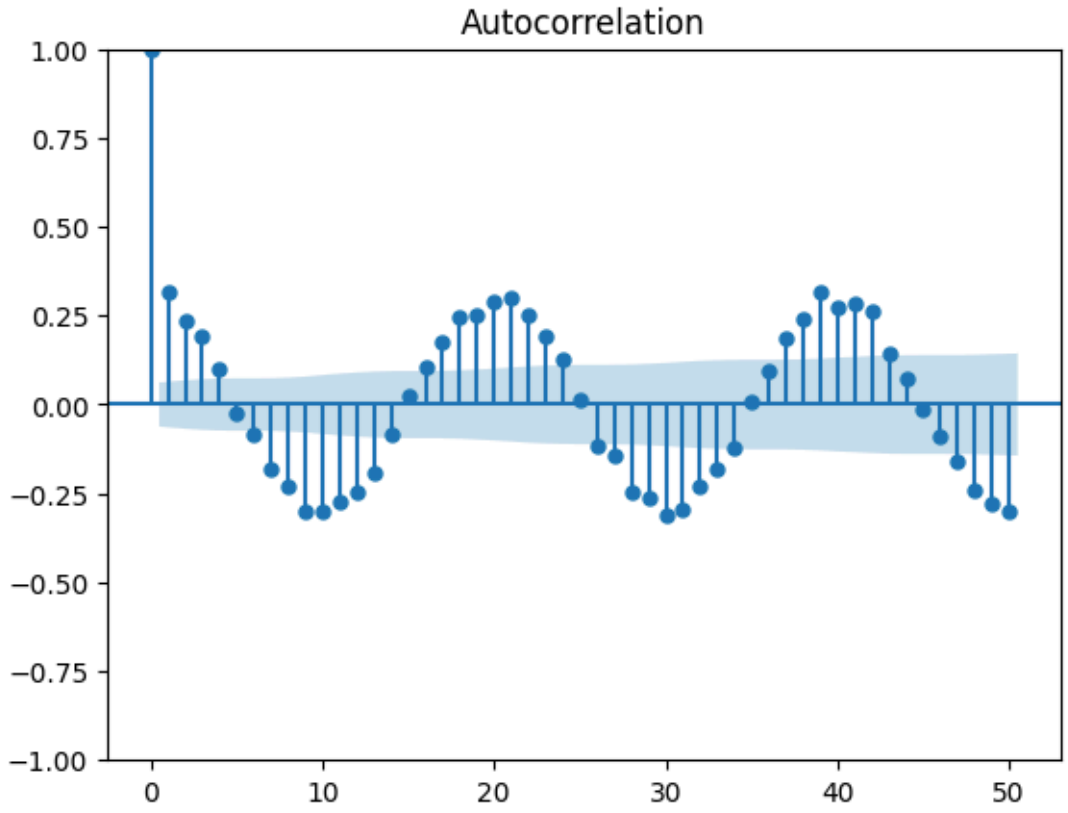
So the ACF graph for lags 0 to 50 gives:

```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(X, lags=np.arange(51))
plt.show()
```

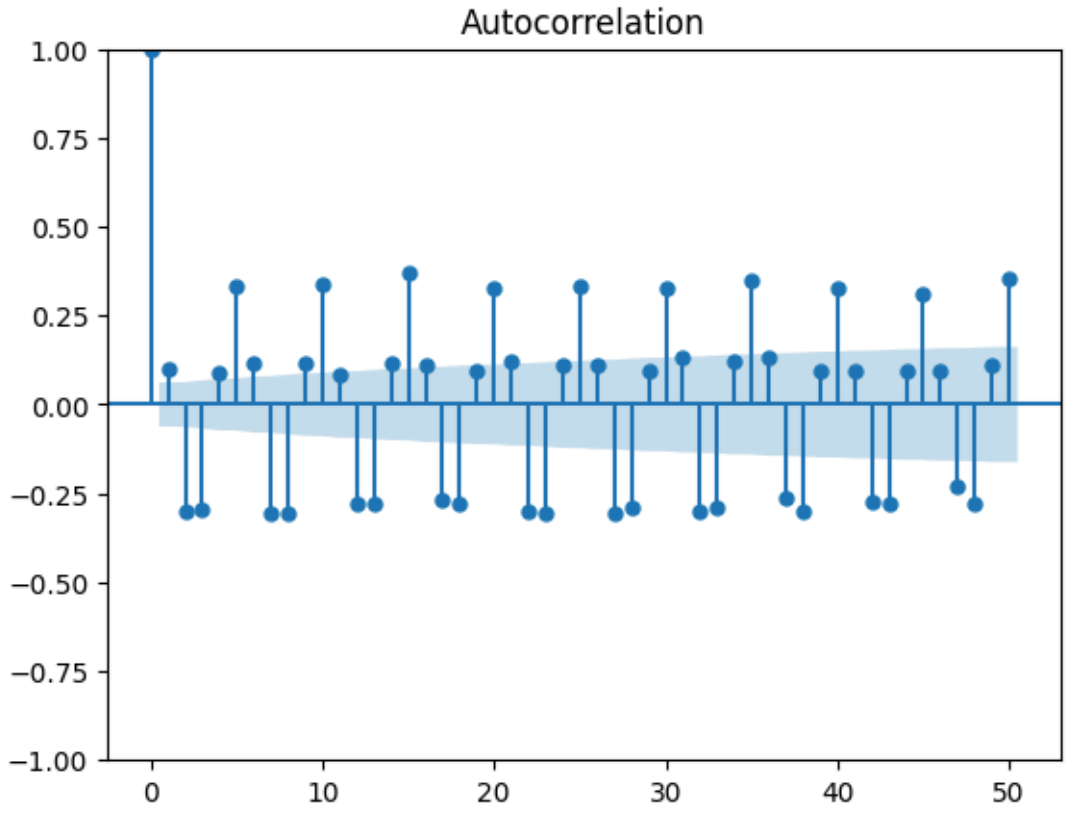


1. Try with other values of T and comment on the impact of T on the ACF graph.

```
plot_acf(X_t(np.arange(1, 1001), 20), lags=np.arange(51))
plt.show()
```




```
plot_acf(X_t(np.arange(1, 1001), 5), lags=np.arange(51))
plt.show()
```



We have tried to plot ACF graph for $T' = 2T$ and for $T' = \frac{T}{2}$. We can observe that for the plot obtained with $T' = 2T$, the period is two times bigger than the original, and with $T' = \frac{T}{2}$, the period is two times smaller than the original. So it means that T control the period of the ACF.

3 AR model on an empirical time series

The file `eur_usd.txt` (download it from Moodle) contains the daily EUR/USD price evolution with the following data structure:

```
timestamp price
```

where `timestamp` denotes the time (in seconds) measured starting from 01 Jan 1970 00:00:00.000.

You can use the following snippet of Python code to load the data:

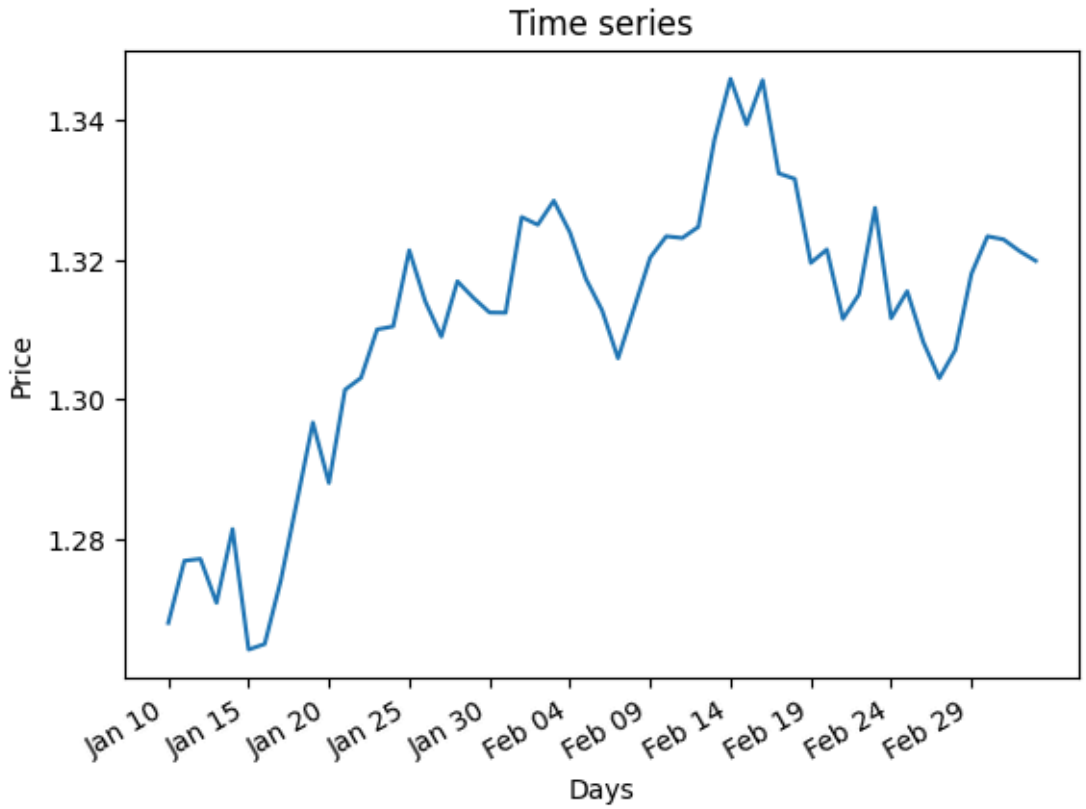
```
from datetime import datetime
import numpy as np
data = np.loadtxt('eur_usd.txt')
```

```
price_ts = data[:, 1]
# Tip: You can convert the timestamp to a better format:
days = [datetime.fromtimestamp(x).strftime('%b %d') for x in data[:, 0]]
```

1. Preprocessing of the time series.

(a) Plot the time series. Does it look stationary?

```
plt.figure()
plt.title("Time series")
plt.plot(days, price_ts)
plt.xlabel("Days")
plt.ylabel("Price")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.show()
```

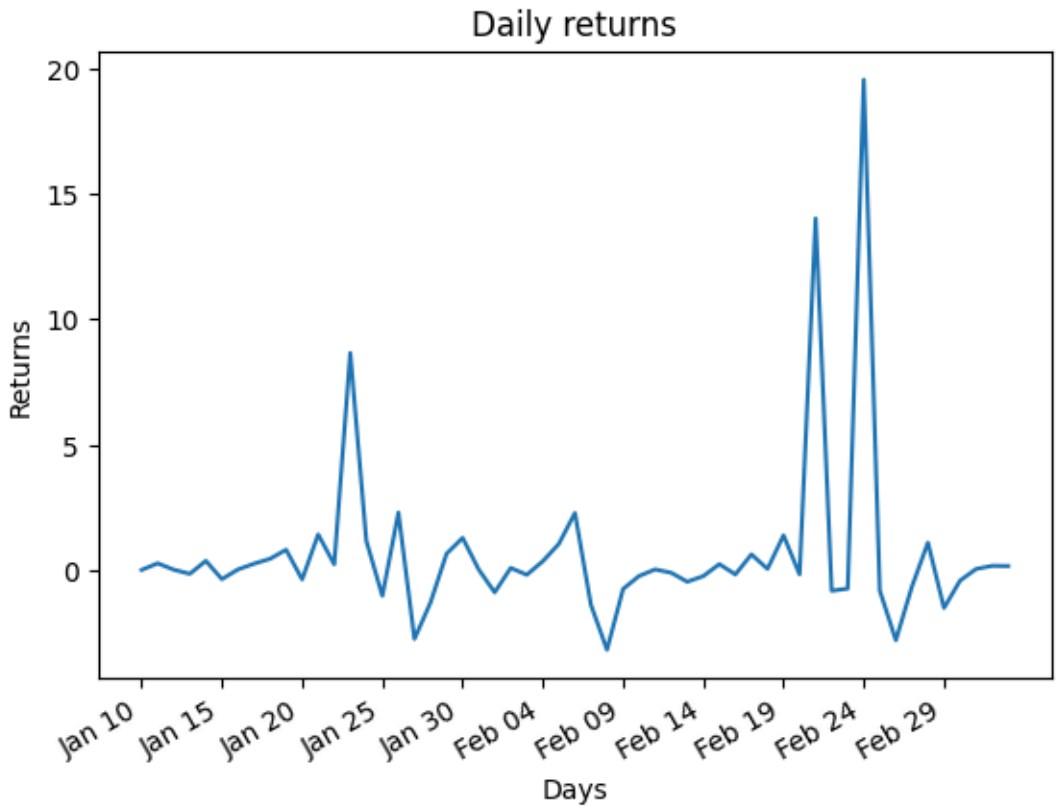


The time series don't look stationary. Indeed, the time series seems to increase over time.

(a) Compute the corresponding daily returns and plot this new time series. Does it look stationary? From now on subtract the mean of this series to center it around zero.

```
def daily_returns(X: np.ndarray) -> np.ndarray:
  returns = (np.roll(X, 1) - X)/X
  returns[0] = 0.
  return returns
```

```
# Center the time series
X = price_ts - np.mean(price_ts)
returns = daily_returns(X)

plt.figure()
plt.title("Daily returns")
plt.plot(days, returns)
plt.xlabel("Days")
plt.ylabel("Returns")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.show()
```

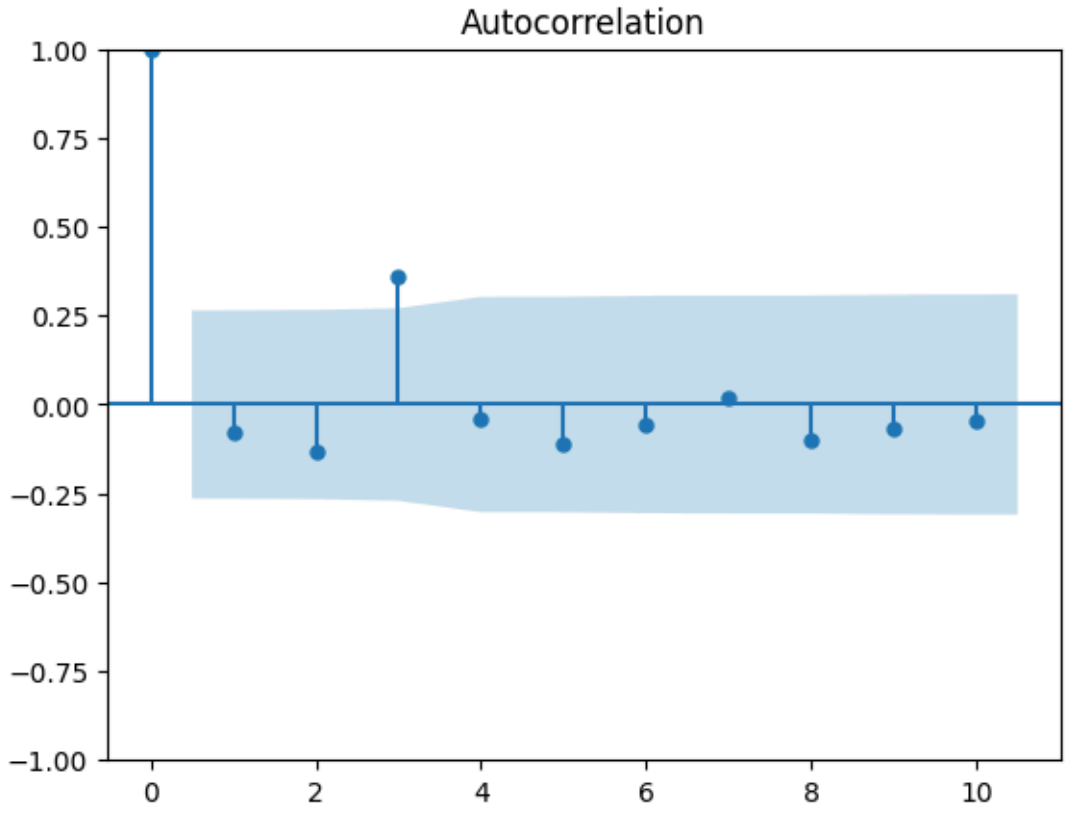


The corresponding daily returns look stationary. Indeed, the mean seems to be constant and the values varies around the mean with some fixed variance.

2. Analysis of the time series of daily returns.

(a) Compute and plot the ACF for lags 0 to 10.

```
plot_acf(returns, lags=np.arange(11))
```



(a) Using the analytical expressions seen during the course, compute the parameter φ_1 of the AR(1) model for this time series.

We have:

$$\varphi_1 = \frac{\mathrm{Cov}(X_t, X_{t+1})}{\mathrm{Var}(X_t)} = \rho_1$$

with ρ_1 the autocorrelation coefficient at lag 1.

So we have:

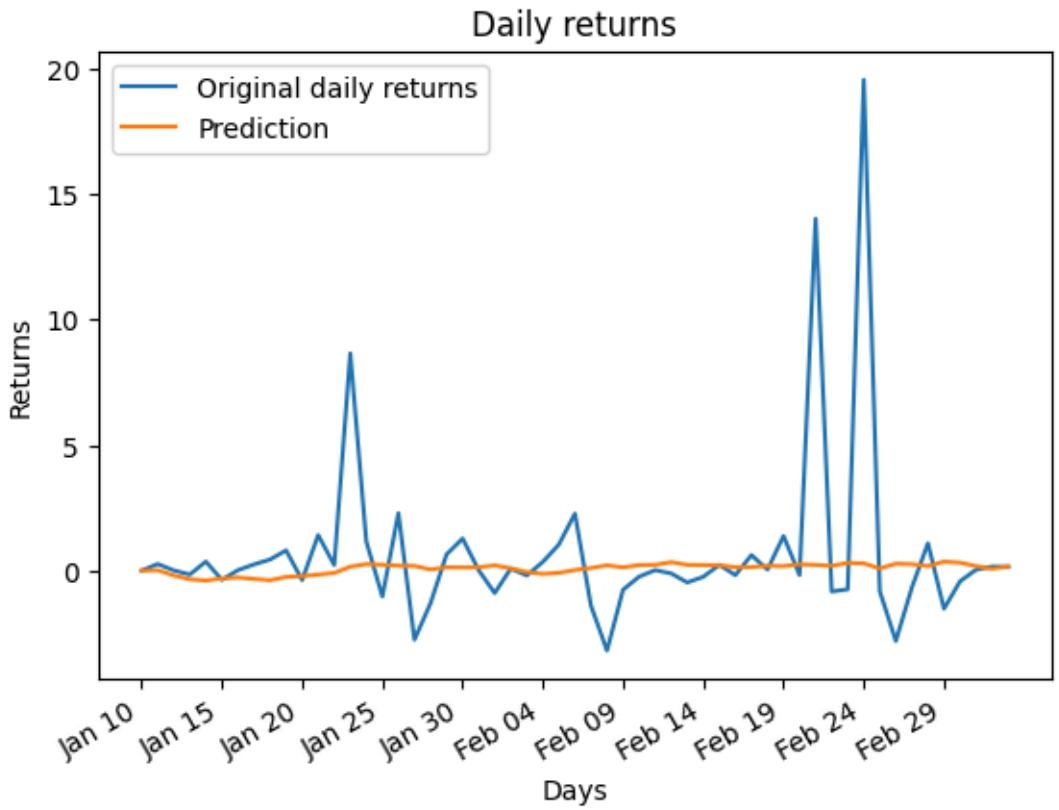
```python
from statsmodels.tsa.stattools import acf

phi_1 = acf(X)[1]
print(f"phi_1 = {phi_1}")
```

```
phi_1 = 0.8783097976730934
```

(a) Use your model to do predictions from the initial value. Plot the predictions along the time series of daily returns on the same graph. What do you think of these predictions?

```python
def AR(phi: float, x_t: float, size: int, std_noise: float) -> np.ndarray:
  x = [x_t]
  for i in range(size - 1):
    x.append(x[-1] * phi + np.random.normal(0, std_noise))
```

```
  return np.array(x)

plt.figure()
plt.title("Daily returns")
plt.plot(days, returns, label="Original daily returns")
plt.plot(days, AR(phi_1, returns[0], len(returns), 0.1), label="Prediction")
plt.xlabel("Days")
plt.ylabel("Returns")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.legend()
plt.show()
```



These predictions are very bad. Indeed, it seems to be some random walk that predict nothing about what will happen. It's because there is not enough information used to perform prediction: taking only one value to predict the next value don't allow to make a good prediction.

To increase the accuracy of the prediction, more values must be taken into account.

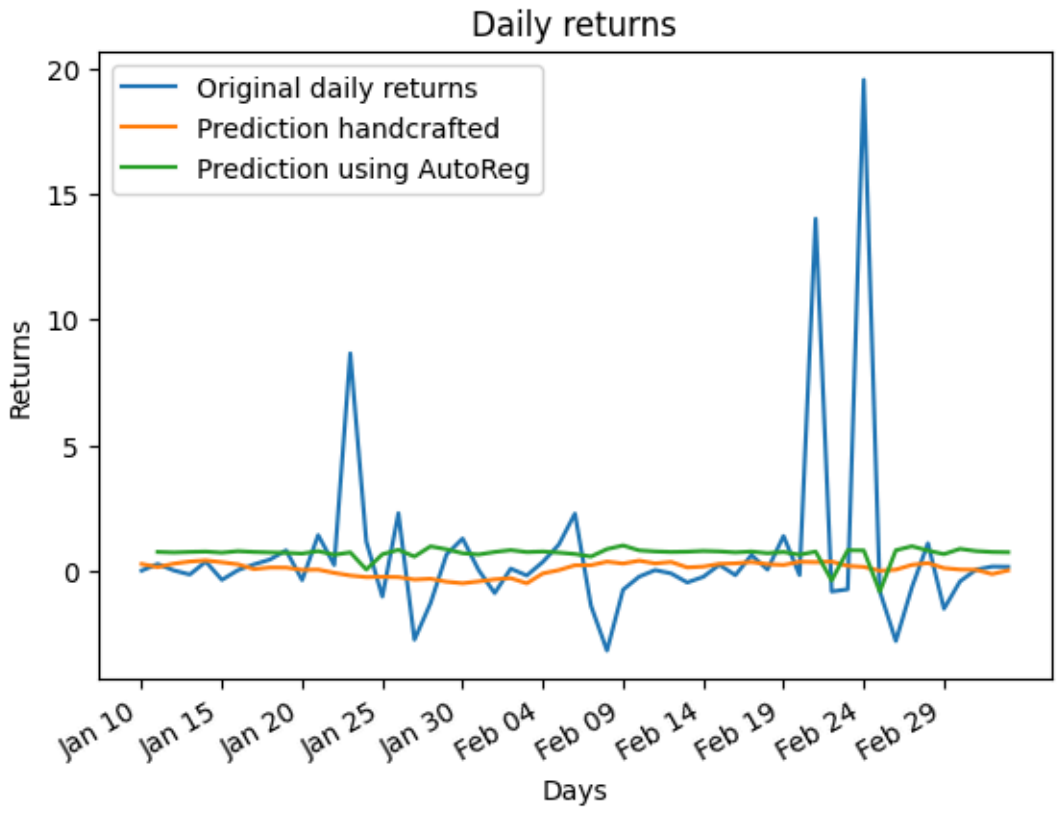
3. AR(p) model with a library.

(a) In Python, you can use the `statsmodels` library to fit an AR(p) model on a time series.

```
from statsmodels.tsa.ar_model import AutoReg
predictions = AutoReg(your_time_series, lags=p).fit().predict()
```

Plot the AR(1) predictions computed with this library and compare it to the predictions of your model obtained in 2c (hint: it should be really similar).

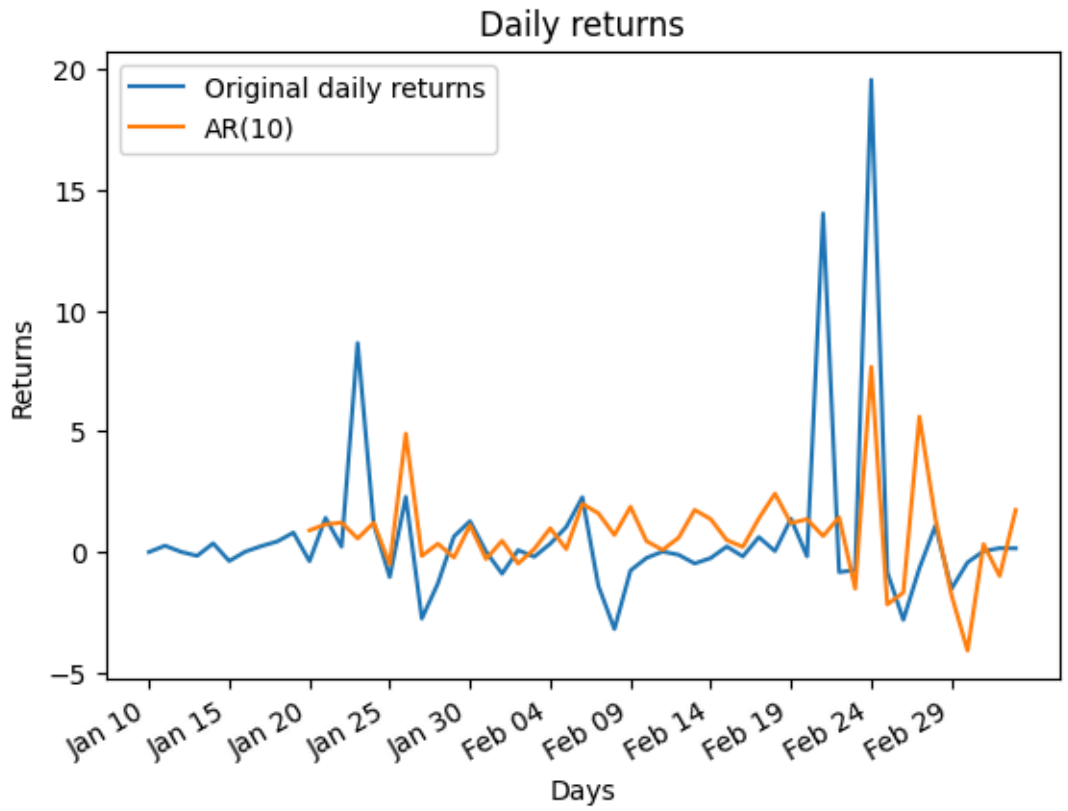
```
from statsmodels.tsa.ar_model import AutoReg

predictions = AutoReg(returns, lags=1).fit().predict()

plt.figure()
plt.title("Daily returns")
plt.plot(days, returns, label="Original daily returns")
plt.plot(days, AR(phi_1, returns[1], len(returns), 0.1), label="Prediction
handcrafted")
plt.plot(days, predictions, label="Prediction using AutoReg")
plt.xlabel("Days")
plt.ylabel("Returns")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.legend()
plt.show()
```

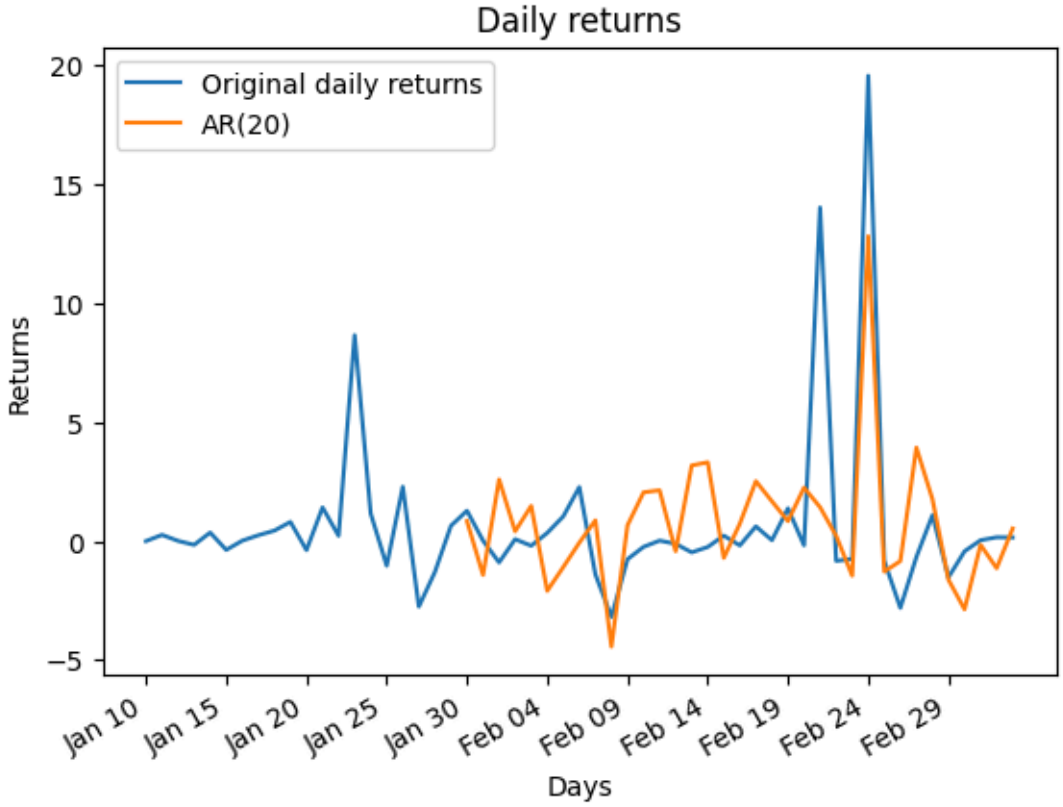


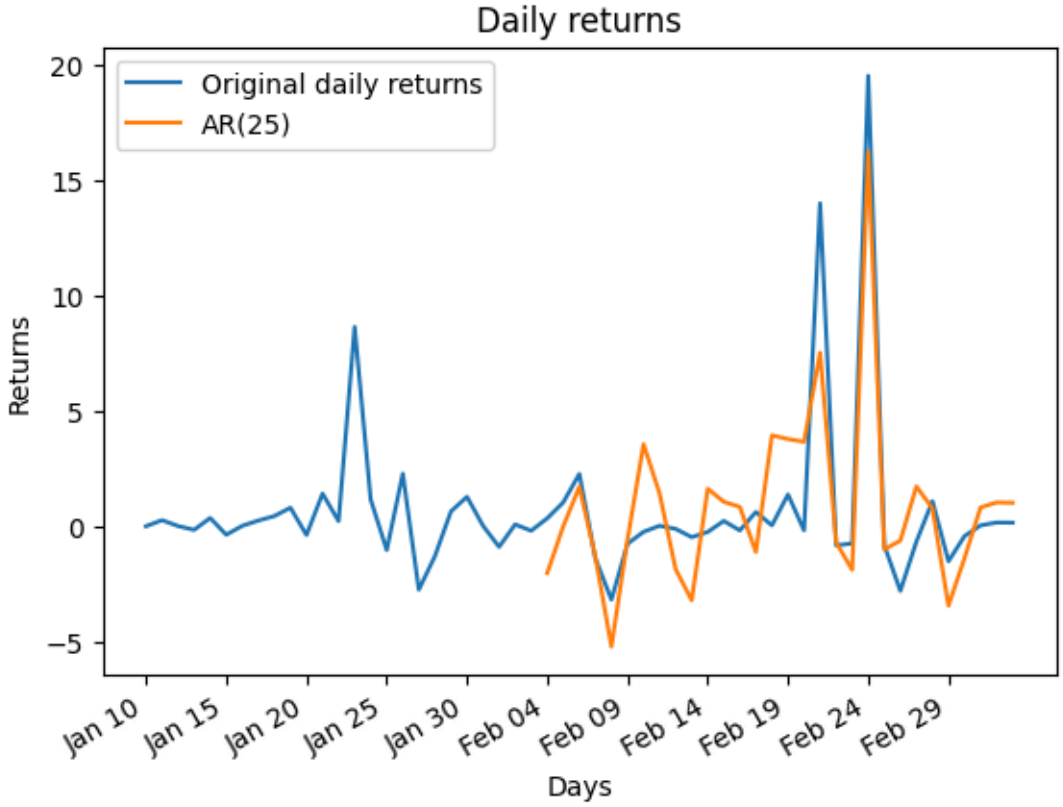
The AR(1) predictions computed with the library looks similar to the prediction obtained by my model.

(a) Using this library, plot the predictions for higher-order AR models. Does the quality of the predictions improve?

```python
for i in [10, 20, 25]:
  plt.figure()
  plt.title("Daily returns")
  plt.plot(days, returns, label="Original daily returns")
  plt.plot(days, AutoReg(returns, lags=i).fit().predict(), label=f"AR({i})")
  plt.xlabel("Days")
  plt.ylabel("Returns")
  plt.xticks((plt.xticks()[0])[::5])
  plt.gcf().autofmt_xdate()
  plt.legend()
  plt.show()
```




Daily returns
Original daily returns
AR(20)
Returns
Days
20
15
10
5
0
−5
Jan 10
Jan 15
Jan 20
Jan 25
Jan 30
Feb 04
Feb 09
Feb 14
Feb 19
Feb 24
Feb 29


Daily returns
Original daily returns
AR(25)
Returns
Days
20
15
10
5
0
−5
Jan 10
Jan 15
Jan 20
Jan 25
Jan 30
Feb 04
Feb 09
Feb 14
Feb 19
Feb 24
Feb 29

With higher-order AR models, the quality of the predictions improve. Indeed, the predictions curve are closer to the original curves. For instance, the prediction predict correctly the big pick that appears on the daily returns. And for AR(25), the prediction of the pick is close to the real value of the pick whereas for AR(10), the pick is predicted, but the magnitude of the pick is too small.

4 FX risk management: EWMA volatility and 1-day VaR

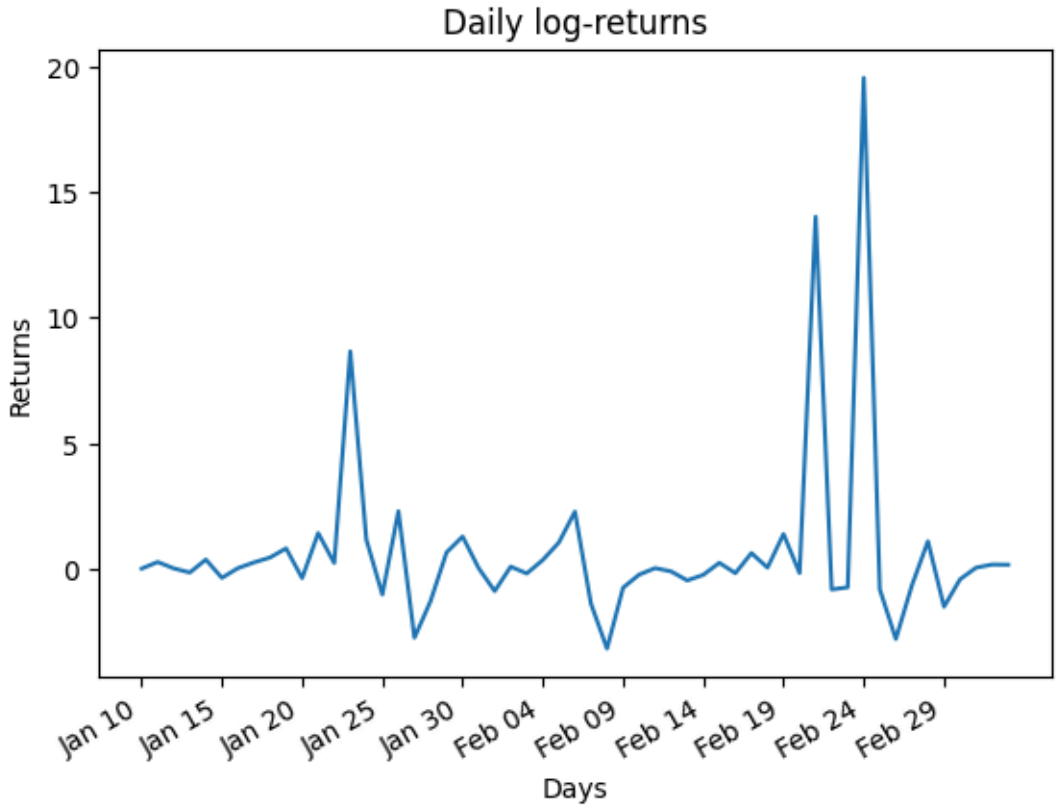
A bit of introduction before delving into the exercise:

- **FX risk:** If you hold a position in a foreign currency (e.g., you are long EUR/USD), you are exposed to FX risk, which is the risk that exchange rate movements will lead to losses.
- **EWMA volatility:** The Exponentially Weighted Moving Average (EWMA) model estimates volatility by giving more weight to recent returns. The formula $\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2$ updates the variance estimate at time t based on the previous variance and the most recent return.
- **VaR:** Value-at-Risk (VaR) is a risk measure that quantifies the potential loss in value of a portfolio over a defined period for a given confidence level. For example, a 1-day 99% VaR of X means there is a 1% chance that the loss will exceed X in 1 day.
- **Rolling 1-day forecast:** This means that each day, you compute a new 1-day-ahead forecast for VaR using all available data up to that day. As new data comes in, your estimates will update, reflecting the latest market conditions.

You will now have to build a simple EWMA model and translate it into a 1-day VaR forecast, then evaluate how often losses breach the VaR threshold. Assuming you hold an EUR/USD exposure, use the daily EUR/USD prices in `eur_usd.txt` to:

1. Compute the daily log-returns $r_t = \ln(P_t/P_{t-1})$ and plot the series.

```
def daily_log_returns(X: np.ndarray) -> np.ndarray:
  log_returns = np.log(np.roll(X, 1)/X)
  log_returns[0] = 0.
  return log_returns

log_returns = daily_log_returns(price_ts)

plt.figure()
plt.title("Daily log-returns")
plt.plot(days, returns, label="daily log-returns")
plt.xlabel("Days")
plt.ylabel("Returns")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.show()
```

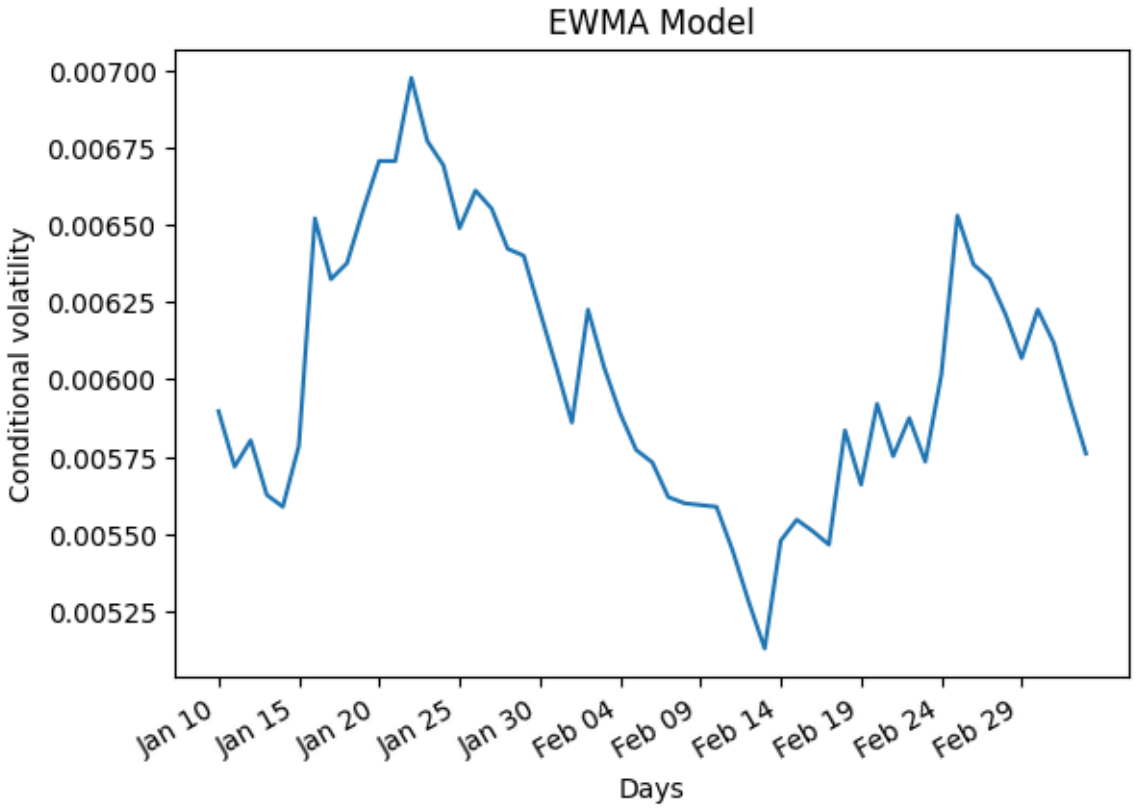


1. Estimate the conditional volatility with an EWMA model:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2$$

using $\lambda = 0.94$. Initialize σ_0^2 with the sample variance of the return series. Plot σ_t.

```
def ewma(r: np.ndarray, l: float = 0.94):
  sigma = [np.var(r)]
  for i in range(1, len(r)):
    sigma.append(l * sigma[i-1] + (1 - l) * r[i - 1]**2)
  return np.array(sigma)

plt.figure()
plt.title("EWMA Model")
plt.plot(days, np.sqrt(ewma(log_returns)))
plt.xlabel("Days")
plt.ylabel("Conditional volatility")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.show()
```

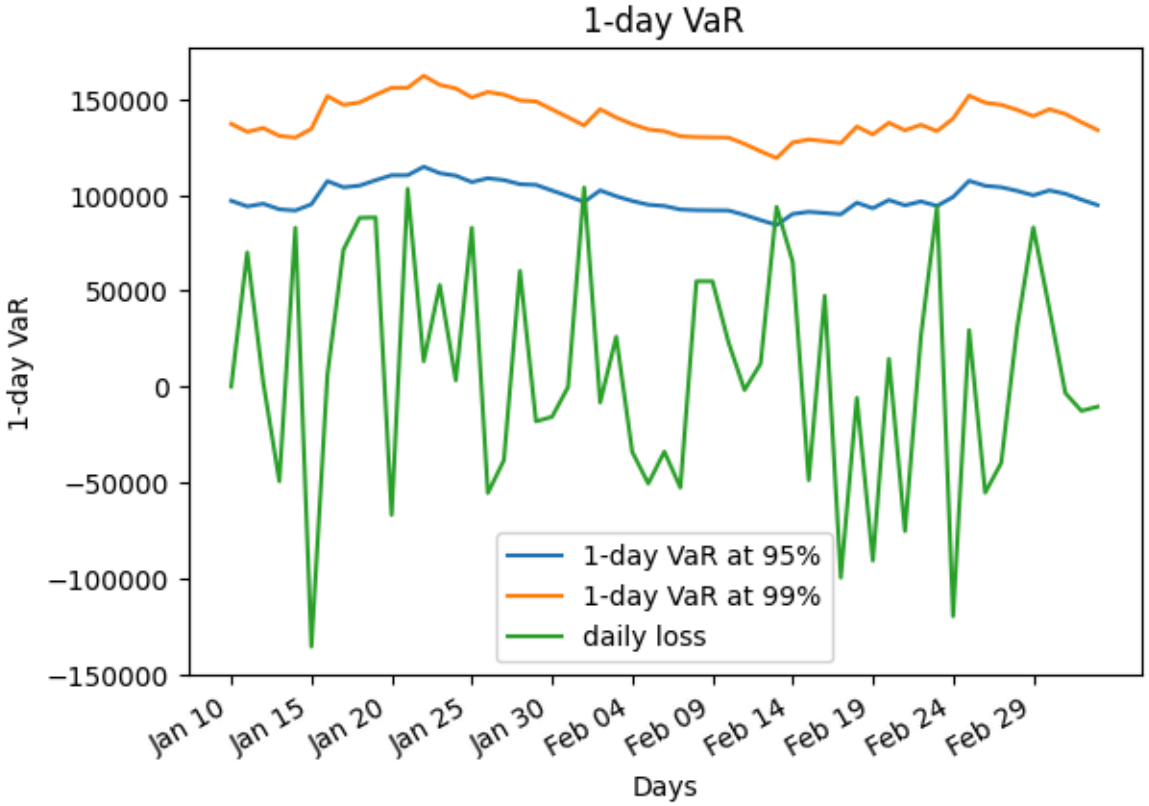


1. Assume a position value of $V_0 = 10'000'000$ USD. Compute the 1-day VaR at 99% and 95%:

$$\mathrm{VaR}_t^{\alpha} = z_\alpha \sigma_t V_0, \quad z_{0.99} = 2.326, \quad z_{0.95} = 1.645$$

where z_α is the standard normal quantile satisfying $P(Z \leq z_\alpha) = \alpha$ for $Z \sim \mathcal{N}(0,1)$. Plot the daily loss $L_t = -V_0 r_t$ together with $\mathrm{VaR}_t^{99\%}$ and $\mathrm{VaR}_t^{95\%}$.

```
V_0 = 1e7
z_99 = 2.326
z_95 = 1.645
plt.figure()
plt.title("1-day VaR")
plt.plot(days, z_95 * V_0 * np.sqrt(ewma(log_returns)), label="1-day VaR at
95%")
plt.plot(days, z_99 * V_0 * np.sqrt(ewma(log_returns)), label="1-day VaR at
99%")
plt.plot(days, -V_0 * log_returns, label="daily loss")
plt.xlabel("Days")
plt.ylabel("1-day VaR")
plt.xticks((plt.xticks()[0])[::5])
plt.gcf().autofmt_xdate()
plt.legend()
plt.show()
```



1. **(Optional)** Backtest the VaR: since σ_t uses r_{t-1}, interpret VaR_t as a 1-day-ahead forecast for day t. Compute the fraction of days such that $L_t > \mathrm{VaR}_t^{99\%}$ and $L_t > \mathrm{VaR}_t^{95\%}$. Compare to the expected exceedance rates (1% and 5%) and report the expected number of exceedances (n_α) for the sample size. Comment on the impact of a short sample.

> **Note on the interpretation:** VaR backtests are noisy on short samples. If you have only a few dozen observations, it is entirely plausible to observe zero 99% exceedances even when the model is correct. This exercise is about the workflow and interpretation, not about passing a formal backtest.